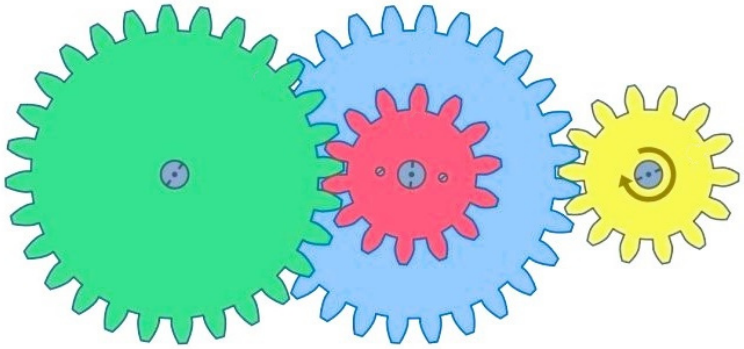
Problem A. Axles and Gears

Source file name:	Axles.c, Axles.cpp, Axles.java, Axles.py
Input:	`Standard`
Output:	`Standard`

You have an assortment of circular gears with varying numbers and sizes of teeth. You also have a motor that spins at one revolution per second, as well as an unlimited number of (identical, arbitrarily long) axles. The motor and all gears fit the axles, and everything attached to a particular axle rotates at the same angular speed. Two gears with the same size teeth can be enmeshed with each other. Gears with different size teeth cannot be enmeshed with each other (though they can be placed on the same axle).



You can arrange the gears and axles in any order. What is the maximum rate at which the last gear/axle in sequence spins that you can achieve? Because this may be large, output the *natural log* of the value.

Input

The first line of input contains a single integer n ($0 \leq n \leq 10^5$) denoting the number of gears.

Each of the next n lines contains two integers s ($1 \leq s \leq 10^5$) and c ($3 \leq c \leq 10^5$), one for each gear in your collection, where s is the size of the teeth of the gear and c is the count of the number of teeth.

Output

Output a single line with a single number equal to the *natural log* of the maximum angular speed you can achieve with your motor and axles and gears in your collection. This output will be considered correct if it is within an absolute or relative error of 10^{-6} .

Example

Input	Output
6 19 364 21 1023 19 66 19 242 21 807 19 675	2.9704451880078357
4 33 10 33 27 44 10 44 27	1.9865035460205664

Problem B. Bikes and Barricades

Source file name:	Bikes.c, Bikes.cpp, Bikes.java, Bikes.py
Input:	`Standard`
Output:	`Standard`

Scott wants to ride his bike along a straight road. But the road has some barricades! Scott will ride his bike up to the first barricade and stop.

Model Scott's straight road as the positive Y axis, with Scott starting at the origin. The barricades are line segments, specified by their endpoints. Determine how far Scott can ride, or if his path is completely unobstructed.

Input

The first line of input contains a single integer n ($1 \leq n \leq 10^3$), which is the number of barricades.

Each of the next n lines contains four integers x_1, y_1, x_2 and y_2 ($-100 \leq x_1, y_1, x_2, y_2 \leq 10^2$, $x_1 \neq 0, x_2 \neq 0$), representing a barricade that runs from (x_1, y_1) to (x_2, y_2). It is guaranteed that no barricade will run through the origin.

Output

Output a single real number, which is how far Scott can ride before he hits the closest barricade, or -1.0 if no barricades get in Scott's way. This output will be considered correct if it is within an absolute or relative error of 10^{-2}.

Example

Input	Output
2 -10 7 5 19 -1 -1 8 21	1.4444444444444446
2 4 -6 -12 -1 3 5 8 8	-1.0

Problem C. Call for Problems

Source file name:	Call.c, Call.cpp, Call.java, Call.py
Input:	`Standard`
Output:	`Standard`

The Call for Problems for the ICPC North America Qualifier (NAQ) has finished, and a number of problems were proposed. The judges voted on the difficulty of each problem. The NAQ does not want to be considered an odd contest, so therefore they refuse to use any problem which has an odd number (not divisible by two) for a difficulty rating.

Given the difficulty ratings of the candidate problems, how many were excluded by this rule?

Input

The first line of input contains a single integer n ($1 \leq n \leq 50$), which is the number of candidate problems.

Each of the next n lines contains a single integer d ($0 \leq d \leq 100$), which are the difficulty ratings of the n problems.

Output

Output a single integer, which is the number of candidate problems excluded by the rule.

Example

Input	Output
2 2 3	1

Problem D. Delphi Danger

Source file name:	Delphi.c, Delphi.cpp, Delphi.java, Delphi.py
Input:	`Standard`
Output:	`Standard`

The temple of Apollon in Delphi. CC BY-SA 3.0 by Inkey on Wikimedia Commons

Have you ever been walking towards someone, dodged left only to see them do the same, and then entered an awkward shuffle to pass each other? Recently, a paper published at the Greek Commerce and Procurement Conference suggested that this may have already been a common problem in ancient Greece.

Back then, there were n merchant caravans that occasionally passed through the Thermopylae. When two caravans met while travelling in opposite directions, both would simultaneously choose to steer either cliffside or seaside. If they made different choices, they could safely pass each other. However, if both chose the same side, they had another opportunity to choose, repeating this process until they could safely pass. Caravans were unable to recognize each other from a distance and would each follow a deterministic strategy to decide which way to steer. Since steering seaside is much more dangerous, we say that the *risk* of a given strategy is defined as the total number of times it dictates to steer seaside. For example, a caravan may follow a strategy described by the sequence "cliffside, seaside, seaside, cliffside, seaside" and then only "cliffside" from then on. This strategy has a risk of 3.

The caravans convened at the oracle in Delphi to ask for advice on which strategies they should each adopt, but this went about as well as could be expected. Rather than helping, the oracle rattled off m prophecies of the following form: When caravans u and v meet, they will awkwardly steer in the same direction exactly t times before safely passing each other by steering in opposite directions on their $(t+1)$th attempt.

The real strategies were lost to history, but you wonder how risky they must have been. You think it is unwise to go against the oracle (this has ended poorly in the past) and want to minimize the *overall risk*, defined as the sum of the risks of all individual strategies.

Input

The input consists of:

- One line with two integers n and m ($2 \leq n \leq 2 \cdot 10^5$, $1 \leq m \leq 4 \cdot 10^5$), the number of caravans and the number of prophecies.
- m lines, each with three integers u, v, and t ($1 \leq u, v \leq n$, $u \neq v$, $0 \leq t \leq 10^9$), describing a prophecy: the strategies of caravans u and v will cause them to steer in the same direction exactly t times, and then steer in opposite directions.

It is guaranteed that each pair of caravans appears at most once.

Output

If there is no set of strategies that can fulfil all the prophecies, output "`impossible`". Otherwise, output "`possible`" followed by the minimum overall risk among all sets of strategies that fulfil the prophecies.

Example

Input	Output
5 3 1 2 2 1 3 0 4 5 2	possible 3
4 3 1 2 3 2 4 4 1 4 2	impossible

Explanation

In the first example, the strategies can be chosen as follows:

- caravan 1 always steers `cliffside` (risk 0)
- caravan 2 steers `cliffside`, `cliffside`, `seaside`, and then `cliffside` from then on (risk 1)
- caravan 3 steers `seaside` and then `cliffside` from then on (risk 1)
- caravan 4 always steers `cliffside` (risk 0)
- caravan 5 steers `cliffside`, `cliffside`, `seaside`, and then `cliffside` from then on (risk 1)

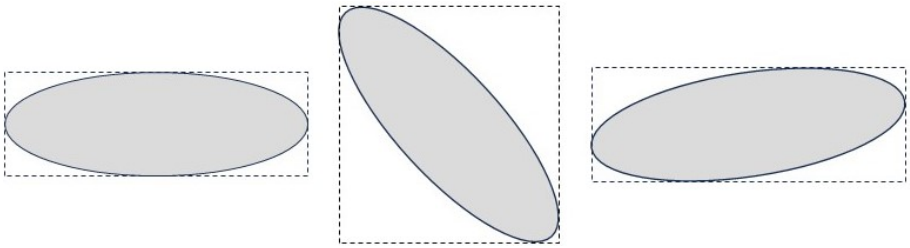
This satisfies all prophecies, yielding a minimum overall risk of 3.

Problem E. Ellipse Eclipse

Source file name:	Eclipse.c, Eclipse.cpp, Eclipse.java, Eclipse.py
Input:	`Standard`
Output:	`Standard`

An *Ellipse* is a 2D geometric figure. It consists of the set of all points such that the sum of the distance from each point to two specific points is constant. The two specific points are called the Foci of the ellipse. The *Major Axis* of an ellipse is the diameter of the ellipse that runs through both foci. It is the longest diameter of the ellipse.

Given an ellipse's two foci and the length of its major axis, determine the coordinates of the tightest axis-aligned bounding box that can block out that ellipse.



Input

The single line of input contains five integers x_1, y_1, x_2, y_2 ($-100 \leq x_1, y_1, x_2, y_2 \leq 100$) and a $\left(\sqrt{p(x_2 - x_1)^2 + (y_2 - y_1)^2} \leq a \leq 10^3\right)$, where the two foci of the ellipse are at (x_1, y_1) and (x_2, y_2) and the length of the major axis is a (Note that's the major axis, not the semi-major axis or the major radius).

Output

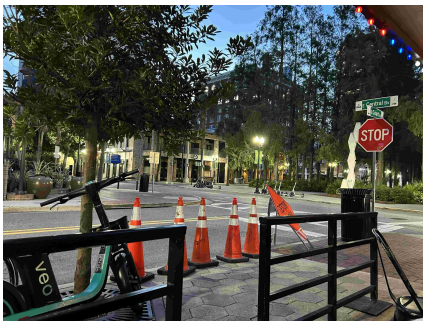
Output four space-separated real numbers on a single line. The numbers, in order, are x_{low}, y_{low}, x_{high}, y_{high}, where $(x_{\text{low}}, y_{\text{low}})$ is the lower left corner of the tightest bounding box of the ellipse, and $(x_{\text{high}}, y_{\text{high}})$ is the upper right corner of the tightest bounding box of the ellipse. Each output will be considered correct if it is within an absolute or relative error of 10^{-4}.

Example

Input	Output
-5 0 5 0 16	-8.000000 -6.244998 8.000000 6.244998
51 23 19 67 70	7.778685 13.871235 62.221315 76.128765

Problem F. Fighting Fraud

Source file name:	Fraud.c, Fraud.cpp, Fraud.java, Fraud.py
Input:	`Standard`
Output:	`Standard`



Picture of a GCPC courier. They are extremely discreet.

Got a job for you. This shady business called *Guaranteed Clandestine Private Couriers* is trying to hide something. Their public filings and tax records are spotless. As clean as a doctor's hands just before surgery. And you know what that means. They had dirt to scrub off, I'm sure of it. Got my hands on one of their drivers' delivery schedules. You know, a list of tasks to discreetly pick up and deliver an item. I bet we find something off with the schedule. Maybe an item that is picked up but never delivered. Maybe a delivery of something with no records where it came from. Heck, they might even list an item for some task long after the item has been delivered to keep the actual cargo off the books. Anyway, find out if the schedule is legit. If each item mentioned in the schedule is picked up and delivered exactly once, then the guy who cooks their books must be a genius. Guess they are paying him well. Speaking of, this one's double your regular fee. Half upfront for your discretion and the rest when I have your report. Don't call unless you're done!

Input

The input consists of:

- One line with an integer n, the length of the schedule ($1 \leq n \leq 10^5$).
- The next n lines each describe a schedule task, the ith of which contains:
  - The string "`pickup`" or "`dropoff`".
  - A string a_i consisting of English lowercase letters (`a-z`), the identifier of the item related to the ith task ($1 \leq |a_i| \leq 20$, $\sum |a_i| \leq 10^6$).

Output

Output "`yes`" if the schedule is valid and "`no`" otherwise.

Example

Input	Output
4 pickup apples pickup oranges dropoff oranges dropoff apples	yes
6 pickup gcpc pickup teams dropoff gcpc pickup jury dropoff teams dropoff gcpc	no
2 dropoff books pickup books	no
4 pickup balloon dropoff balloon pickup balloon dropoff balloon	no
1 dropoff fft	no
4 pickup timelimit pickup correct dropoff timelimit dropoff correct	yes

Problem G. Genetic Reconstruction

Source file name:	Genetic.c, Genetic.cpp, Genetic.java, Genetic.py
Input:	`Standard`
Output:	`Standard`

You're in charge of studying a new species, and you'd like to make sure the work that you've collected is correct.

There are several creatures. For each one, you know the eye color they have. This is denoted by one of the first 20 English lowercase letters (from '`a`' to '`t`').

You think that there is a gene that controls eye color. Your hypothesis is that each creature has two *Alleles*. An allele is represented by a lowercase English letter. The resulting eye color of the creature is the allele of the two which comes first alphabetically.

In addition, you've identified two parents of each creature. Some parents' information may be missing; either both parents' information will be available or both will be missing. A creature inherits one allele from each parent; any of the four combinations is possible. For example, one parent with alleles '`ak`' (thus eye color '`a`'), and another with alleles '`em`' (eye color '`e`'), might have a child with alleles '`ae`' (eye color '`a`'), '`am`' (eye color '`a`'), '`ek`' (eye color '`e`') or '`km`' (eye color '`k`').

Given the number of creatures, the parent information, and the eye color of each creature, determine if this information is consistent with your hypothesis.

Input

The first line of input contains a single integer n ($1 \leq n \leq 20$), which is the number of creatures in your study. The creatures are numbered from 1 to n.

Each of the next n lines contains two integers p_1, p_2 ($(1 \leq p_1, p_2 \leq n, p_1 \neq p_2)$ or $p_1 = p_2 = 0$) and a character c ($c \in$ { 'a', ..., 't' }), where p_1 and p_2 represent the parents of the given creature (or are both 0 if the parents are unknown), and c is the given creature's eye color. Note that either both parents will be known or both will be unknown, and that both parents' numbers will be less than the number of the given creature; no creature can be its own ancestor or descendant.

Output

If the information is consistent, output the alleles for each creature one per line, with no spaces; otherwise output -1. If there are multiple possible solutions, output only the one that comes first alphabetically (e.g., the alphabetically first allele pair for first creature, then second, and so on).

Example

Input	Output
3 0 0 a 0 0 b 1 2 c	ac bc cc
3 0 0 c 0 0 c 2 1 a	-1

Problem H. Historical Hits

Source file name:	Hits.c, Hits.cpp, Hits.java, Hits.py
Input:	`Standard`
Output:	`Standard`

Hitster is a tabletop game about placing songs into an ordered *timeline* by guessing their release year. As a longtime member of the *Companionless Playing Card Games (CPCG)* association, Harper has access to a single-player variant of Hitster. The rules are similar to regular Hitster. Harper starts with an empty timeline. In each round of the game, she draws a card which contains only a QR code on its front. She scans the QR code to listen to the song and afterwards guesses the release year of the song. Afterwards, she turns the card to reveal the song's true release year. Suppose that the true year is a and she guessed b. If Harper's timeline does not yet contain a song with a *true* release year that is strictly between a and b, she adds the card to her timeline.[1] Otherwise, she discards the card.

Harper just had a lengthy playthrough of the entire deck of n cards. Despite her thorough knowledge of music, her final timeline is shorter than she expected. Playing a second time now would be boring because Harper remembers the correct year of all the songs, along with her not-so-correct guesses.[2] Surely, she was just unlucky with the order in which she drew the cards, she thinks. To confirm her suspicion, Harper turns to probability theory. Suppose that the same cards are presented in an order drawn uniformly at random from the set of all $n!$ permutations. What is the expected length of Harper's final timeline if she repeats her guesses from her last playthrough for all of the n songs?

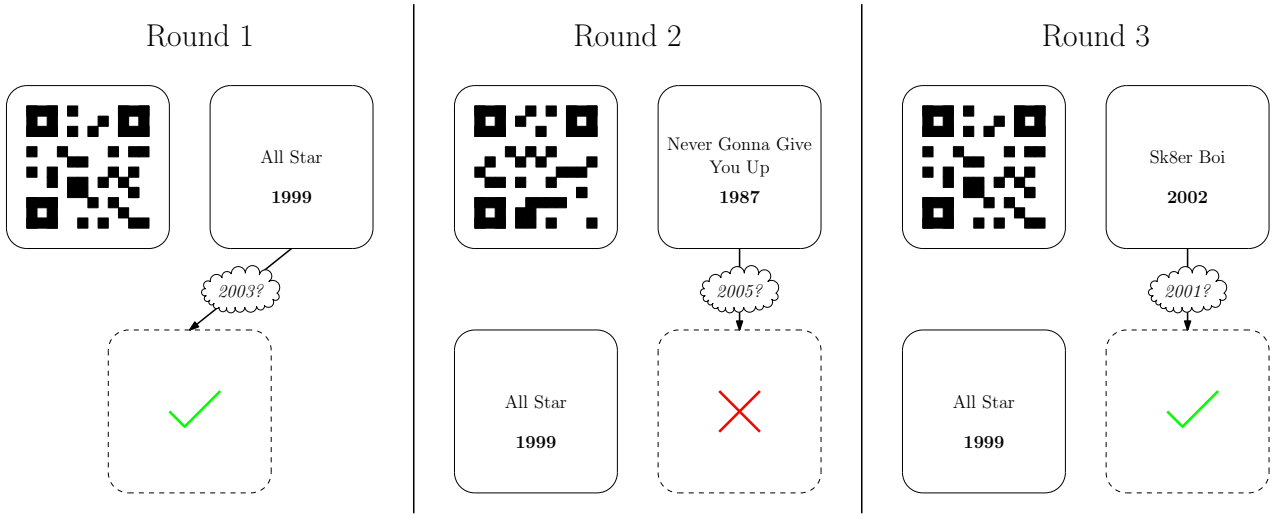


Figure X. Visualization of a playthrough of the game with the cards of Sample 3 presented in the order of the input. Harper's timeline is shown at the bottom. After the third round, her final timeline contains two cards. The top right of each round shows the back of the card, which Harper cannot see before making her guess.

Input

The input consists of:

- One line with an integer n ($1 \leq n \leq 3000$), the number of cards.
- n lines, the ith of which contains two integers a_i, b_i ($1 \leq a_i, b_i \leq 10^9$), the correct year and Harper's guessed year for the song of the ith card.

[1] Her guess b never equals another card's true year, so the card's position in the timeline is uniquely determined.
[2] She developed this incredible memory through many long nights of playing a single-player variant of *Memory*.

It is guaranteed that $a_i \neq a_j$ and $a_i \neq b_j$ for all $i \neq j$.

Output

Determine the expected length of Harper's timeline if the cards are presented in an order drawn uniformly at random from the set of all $n!$ permutations.

Let $M = 998\,244\,353$. The expected length can be represented as an irreducible fraction p/q where q is not divisible by M. Output one integer $p \cdot q^{-1} \bmod M$ (the unique integer x such that $0 \leq x < M$ and $x \cdot q \equiv p \bmod M$).

Example

Input	Output
2 2002 2004 2001 2003	499122178
2 2001 2001 2002 2002	2
3 1999 2003 1987 2005 2002 2001	831870296

Explanation

Example 1: There are two possible orders. If the cards are drawn in the order of the input, then the first card is added to the timeline and the second card is discarded. If the cards are drawn in reverse order, then both cards are added to the timeline. Thus, the expected length is $3/2$. One can verify that $499122178 \cdot 2 \equiv 3 \bmod M$.

Problem I. Incremented Itinerary

Source file name:	Itinerary.c, Itinerary.cpp, Itinerary.java, Itinerary.py
Input:	`Standard`
Output:	`Standard`

You recently started to go running in your free time. You got the shoes, you got the tracking app, and you even picked up your first injury a couple of days ago. Overall, you are really starting to like this new hobby of yours.

So far, your training routine was to run from your office to your home along the streets of your town. The town consists of n intersections, numbered from 1 to n, which are connected by m bidirectional streets of equal length. Your office is located at intersection 1, and your home is at intersection n.

As the computer scientist you are, you were of course following a shortest path. However, studies indicate that you should not just train to complete the same route in a faster time, but also to complete longer routes. Given that the last time you experimented with new training methods did not go so well, you want to take this new approach slowly. Therefore, you are looking for a route from your office to your home that is exactly one street longer than your current route. To keep the run interesting, you also want to avoid visiting any intersection more than once. Determine whether such a new training route exists.

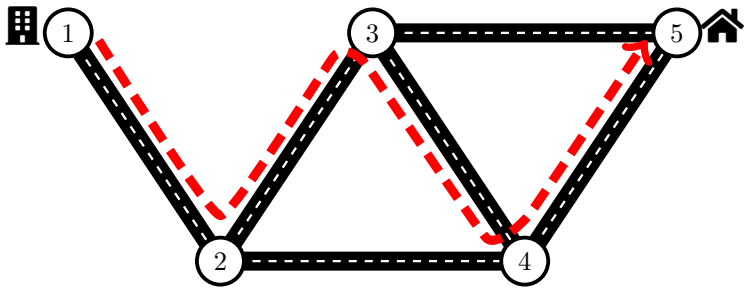


Figure X. Visualization of a possible new training route in the first example.

Input

The input consists of:

- One line with two integers n and m ($2 \leq n \leq 10^5$, $1 \leq m \leq 2 \cdot 10^5$), the number of intersections and the number of streets.
- m lines, each with two integers a and b ($1 \leq a, b \leq n$, $a \neq b$), representing a bidirectional street between intersections a and b.

It is guaranteed that each pair of intersections is connected by at most one street. Further, it is guaranteed that there is at least one route from intersection 1 to intersection n.

Output

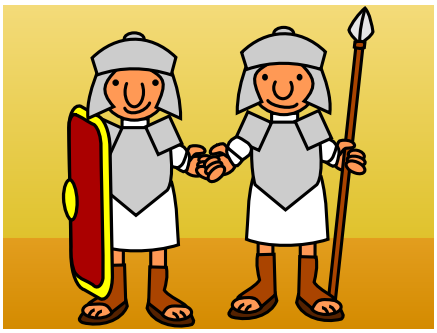
If there is a route that can serve as your new training route, output "`possible`". Otherwise, output "`impossible`".

Example

Input	Output
5 6 1 2 2 3 3 4 4 5 2 4 3 5	possible
2 1 1 2	impossible
6 7 1 2 1 3 2 6 3 6 2 4 4 5 5 6	impossible
6 5 1 3 3 6 6 2 2 5 5 1	possible

Problem J. Junior Joining

Source file name:	Joining.c, Joining.cpp, Joining.java, Joining.py
Input:	`Standard`
Output:	`Standard`



Patrol pair with shield and spear. Public Domain on publicdomainvectors (edited).

Arya, the Commander of Gil'ead, has received $2 \cdot n$ new recruits. Since the Gil'ead City Protectorate Council has mandated more patrols, she intends to pair them into n pairs to patrol the area. To maximize the effectiveness of a patrol pair in the modern era, it should consist of one shield bearer and one spear carrier.

Each recruit i has shield-defense skill d_i, and spear-attack skill a_i. While skilled usage of their respective tool is important, one should never neglect the effectiveness of strong synergy. Because of long-standing rivalries between cities, strong bonds have formed within each city. With enough discipline, Arya is sure that there will not be any bad blood within a pair, but if both are from the same city, they will synergize exceptionally well.

The total fighting strength of a pair is the sum of the shield-defense skill of one, the spear-attack skill of the other, and if both recruits are from the same city c, also a synergy bonus of c.

Determine the maximum sum of fighting strength over all pairs that Arya can achieve with optimal pairing.

Input

The input consists of:

- One line with an integer n ($1 \leq n \leq 10^5$), half the number of new recruits.
- $2 \cdot n$ lines, the ith of which contains three integers d_i, a_i, and c_i ($1 \leq d_i, a_i, c_i \leq 10^6$), the shield-defense skill, spear-attack skill, and the home city of the ith recruit.

Output

Output the maximum achievable sum of fighting strength over all pairs.

Example

Input	Output
1 4 2 1 3 2 2	6
2 1 2 5 4 2 5 5 1 5 3 2 1	18
2 1 7 2 1 5 2 8 1 6 7 1 6	27

Explanation

Example 1: There are only two recruits so they are paired together. Recruit 1 is assigned the shield, recruit 2 the spear. As they are not from the same city, only their skill is counted $4 + 2 = 6$.

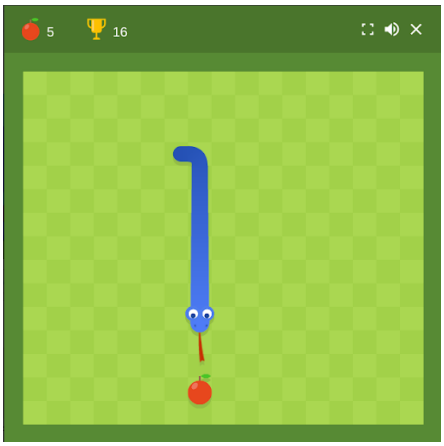
Example 2: Recruit 3 (shield) is paired with recruit 1 (spear). As they are both from city 5, they have 5 bonus strength from synergy ($5 + 2 + 5 = 12$ total). Recruit 2 (shield) and recruit 4 (spear) are paired, resulting in a fighting strength of 6.

Example 3: One possible pairing is the following: recruit 3 (shield) with 1 (spear), and recruit 4 (shield) with 2 (spear).

Problem K. Ksnake ... Snake

Source file name:	Ksnake.c, Ksnake.cpp, Ksnake.java, Ksnake.py
Input:	`Standard`
Output:	`Standard`

Snake is a video game classic, preserved at the Museum of Modern Art (MoMA) and listed as one of the "Top 100 Video Games" of all time. The goal of the game is to move a snake's head to an apple. Once the snake reaches the apple, it eats it and grows in length. A new apple is placed, which the now grown snake must then eat.



Google's version of Snake

The game is played on a grid, and every segment of the snake's body occupies one cell. The snake's head can turn in three directions, but it cannot go backwards. The body follows the head. The head may not collide with the body or exit the grid. Since the entire snake moves at the same time, the head is allowed to enter the cell that the tail is vacating.

Playing the game requires quickness and foresight. It's all too easy to take turns that put the snake head in a position where it's doomed to hit the wall or its body before reaching the apple, especially as the snake grows longer.

You're being asked to write a program that can determine whether the snake's head can reach the apple from a given position, or not and the snake is doomed to die.

Input

The first line of output contains two integers r and c ($1 \leq r, c \leq 10,\ r \cdot c \geq 2$), where the grid has r rows and c columns.

Each of the next r lines contains a string of length exactly c characters from the set

$$\{\ \text{`.'}, \text{`0'}, \ldots, \text{`9'}, \text{`a'}, \ldots, \text{`f'}, \text{`A'}\ \}$$

where '.' represents an open cell in the grid, the hexadecimal digits '0', ..., '9' and 'a', ..., 'f' represent the snake, and 'A' represents the apple. The snake may be anywhere from one to sixteen characters long, with '0' as its head, followed by the other hexadecimal digits in strict order ('1' follows '0', '2' follows '1', etc., with no skipping digits.). It is guaranteed that there is at most one of each digit, each digit (except '0') is adjacent to the immediately previous digit, and that there is exactly one apple in the grid.

Output

Output a single integer, which is 1 if the snake can reach the apple, and 0 if it cannot and is doomed to die.

Example

Input	Output
5 8 ......01 ....98.2 ...A.7.3 .....654	1
5 4 ...A .... 6789 5432 ..01	0
5 5 ....A ..... 678.. 54321 0	1
4 4 567A 4389 12ba 0dc.	1

Problem L. Lottery

Source file name:	Lottery.c, Lottery.cpp, Lottery.java, Lottery.py
Input:	`Standard`
Output:	`Standard`

You suspect your local lottery of cheating! Some numbers are coming up too often!

Each week, the lottery randomly selects five numbers in the range from one to fifty. So, each number should appear about 10% of the time if the numbers are truly chosen randomly. If a number appears far more than that, it's suspicious!

Determine if a set of lottery drawings is suspicious by listing all numbers that appear too often. To allow for random error, you'll need to flag any number that appears more than 20% of the time.

Input

The first line of input contains a single integer n ($1 \leq n \leq 10^3$). There will be $10 \cdot n$ lottery results for you to analyze.

Each of the next $10 \cdot n$ lines contains 5 integers x ($1 \leq x \leq 50$). Each line represents a drawing. All values on a line are unique.

Output

On a single line, output all numbers that appear strictly more than $2 \cdot n$ times in the list. If there is more than one, output them space-separated, in sorted order from smallest to largest. If there aren't any, output -1.

Example

Input	Output
1 32 30 16 45 27 34 45 35 31 42 1 12 26 50 13 34 50 36 21 39 47 7 41 18 45 28 48 2 8 4 16 40 17 2 19 50 4 30 15 6 31 13 33 46 18 49 23 24 17 48	45 50

Problem M. Mirror Magic

Source file name:	Mirror.c, Mirror.cpp, Mirror.java, Mirror.py
Input:	`Standard`
Output:	`Standard`

Mia and Mark both own a chandelier, each of which has n candleholders. After the installation, Mia wonders whether she can hide a part of the room behind a vertically placed mirror for a magic trick. Of course, the mirror should be placed such that Mark would not notice its existence. She believes that she can ensure that Mark will not be able to see her or himself in the mirror, and that she can use clever lighting to hide any possible weirdness resulting from mirroring the room's walls. What worries her most are the chandeliers. Mark knows the precise positions of all the candleholders and would immediately notice if the chandelier in the mirror looked different from the chandelier behind the mirror he expects to see. Naturally, all candleholders of one chandelier should lie on one side of the mirror, and all candleholders of the other chandelier on the other side.

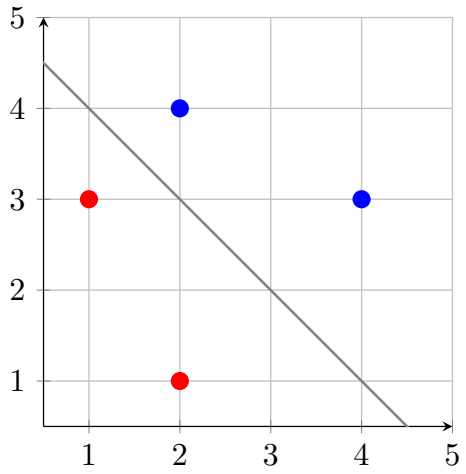


Figure X. Illustration of the third example.

Input

The input consists of:

- One line with an integer n ($1 \leq n \leq 10^5$), the number of candleholders in each chandelier.
- n lines, each containing two integers x_i and y_i ($-10^6 \leq x_i, y_i \leq 10^6$), the coordinates of the ith candleholder of Mia's chandelier.
- n lines, each containing two integers x_i and y_i ($-10^6 \leq x_i, y_i \leq 10^6$), the coordinates of the ith candleholder of Mark's chandelier.

It is guaranteed that all $2 \cdot n$ points are pairwise distinct.

Output

Output "`possible`" if it is possible to place the mirror as desired, and "`impossible`" otherwise.

Example

Input	Output
1 0 0 1 1	possible
2 0 0 2 2 1 1 3 3	impossible
2 1 3 2 1 2 4 4 3	possible
2 2 1 1 3 2 4 4 3	possible
3 1 1 3 1 2 4 1 3 3 3 2 0	impossible
3 -1 -1 -2 -2 -2 1 0 0 1 -1 1 2	impossible
3 -1 -1 -2 -2 -2 1 0 1 1 -1 1 2	impossible